



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

AI-Based Football Analytics Using Computer Vision MLOps Tutorial Report

submitted by

Himashree	1RV23AI072
Preetam Baheti	1RV22AI074
Priyansh	1RV23AI076
R Daksharani	1RV23AI077

Artificial Intelligence and Machine Learning

2024-2025

TABLE OF CONTENTS		
Phase 1:Model Development and Foundations		
Chapter 1		
1.1	Problem Statement and Objectives	3
1.2	Project Scope and Evaluation Metrics	4
1.3	Tools and Environmental Setup	5
1.4	Literature Review	9
Chapter 2		
2.1	Understanding MLOps	11
2.2	Risks Analysis	11
2.3	Risk Matrix	13
Chapter 3		
3.1	Data Collection and Exploration Techniques	14
3.2	Data Profiling, cleaning and preprocessing	14
3.3	Feature Extraction and Feature Selection	15
3.4	Model Design Approaches	15
3.5	Governance and Data Version Control	15
Chapter 4		
4.1	Model Building	18
4.2	Training, Validation and Testing	18
4.3	Evaluation Metrics and Visualisation	19
4.4	Experiment Tracking	19
4.5	Interpreting Model Results and Error Analysis	19
Chapter 5		
5.1	Slides	20
Phase 2:Deployment, Monitoring and Governance		
Chapter 6		
6.1	Revisiting Model Performance	21
6.2	Hyperparameter Tuning and Performance	22
6.3	Interpretability	22
6.4	Model Versioning and Registry	22
6.5	Documentation of Model Updates	23
Chapter 7		
7.1	Overview of CI/CD for ML	24
7.2	Tools Setup	24

7.3	Automating Model Training and Testing	25
7.4	Building and Running a Deployment Pipeline	25
7.5	Continuous Integration Hands-On Exercise	25
Chapter 8		
8.1	Model Monitoring Concepts and KPI's	26
8.2	Tools: Prometheus, Grafana, EvidentlyAI	26
8.3	Detecting Concept Drift and Data Drift	27
8.4	Model Retraining and Redeployment	27
8.5	Logging, Alerting and Dashboard Setup	28
Chapter 9		
9.1	Post-Deployment Performance Review	29
9.2	Continuous Improvement via Feedback Loops	29
9.3	Governance and Ethical AI Practices	29
9.4	Documentation: Model Cards, Data Cards	30
9.5	Audit and Review Checklist	30
Appendix		
A.1	List of MLOps tools, Features, Pros and Cons	31
A.2	Tool Installation Guides	35
A.3	Further Reading	37

Chapter 1

Introduction

1.1 Problem Statement and Objectives

1.1.1. Problem Statement

Football match analysis is traditionally carried out through manual video review and expert observation, which is time-consuming, subjective, and difficult to scale. Although broadcast football videos are widely available, extracting structured and meaningful information such as player positions, ball movement, and team formations from raw video data is challenging due to factors such as occlusion, camera motion, varying viewpoints, and lighting conditions. In addition, many existing machine learning approaches focus only on model development and do not address operational challenges such as data versioning, deployment, monitoring, and continuous improvement. Hence, there is a need to develop an AI-based football analytics system that combines computer vision techniques with MLOps practices to ensure scalability, reproducibility, reliability, and maintainability throughout the machine learning lifecycle

1.1.2. Objectives

The primary objective of this project is to implement an **AI-based Football Analytics System**. This system aims to fundamentally transform the traditional, manual process of football match analysis by automating the extraction of tactical and spatial insights from broadcast video footage. The core function of the system is to employ advanced computer vision and machine learning techniques to accurately detect, track, and analyze players and the ball during a football match.

The system focuses on the following key analytical aspects:

1. Player detection and tracking across video frames
2. Ball detection and trajectory analysis
3. Spatial mapping of players onto a standardized football pitch
4. Team-based positional and control analysis

1.2 Project Scope and Evaluation Metrics

1.2.1. Project Scope

This project focuses on developing an AI-based football analytics system that automatically detects and tracks players, referees, and the ball from broadcast football match videos using computer vision techniques. The scope includes training and evaluating deep learning-based object detection models to handle real-world challenges such as occlusion, camera motion, and varying viewpoints. In addition to model development, the project incorporates MLOps practices such as dataset versioning, experiment tracking, monitoring, workflow automation, and environment standardization to ensure reproducibility, scalability, and maintainability. System performance, model accuracy, and resource utilization are continuously monitored to support reliable analytics. The final outcome is a production-aware prototype that demonstrates how computer vision and MLOps can be integrated for scalable football analytics, without addressing advanced tactical decision-making or live in-stadium deployment.

1.2.2. Evaluation Metrics

The performance of the AI-based football analytics system is evaluated using a combination of model accuracy, training behavior, and system-level metrics to ensure both predictive effectiveness and operational reliability.

1. Object Detection Metrics

- Mean Average Precision (mAP@0.5):
Measures detection accuracy at an IoU threshold of 0.5, indicating how well players, referees, and the ball are localized.
- Mean Average Precision (mAP@0.5:0.95):
Provides a stricter evaluation by averaging performance across multiple IoU thresholds.
- Precision:
Evaluates the proportion of correct detections among all predicted detections, controlling false positives.

2. Training and Validation Metrics

- Training Loss:
Tracks model learning and convergence during training.

- **Validation Loss:**
Used to detect overfitting and assess generalization to unseen data.

3. Computational Performance Metrics

- **Training Time per Epoch:**
Measures training efficiency.
- **GPU Memory Usage:**
Evaluates hardware resource consumption.
- **GPU Utilization:**
Assesses compute efficiency during training and inference.

4. MLOps and System Metrics

- **Experiment Reproducibility:**
Ensured through consistent logging of parameters, metrics, and artifacts.
- **Pipeline Reliability:**
Measured by successful execution of automated training and evaluation workflows.
- **Monitoring Stability:**
Continuous tracking of performance metrics to detect degradation over time.

1.3 Tools and Environmental Setup

1.3.1. Tools Used

MLOps stage	Tools	Purpose
Data Management and Versioning	DVC	Track Dataset changes, versions, reproducibility
Experiment Tracking and Model Management	Roboflow	Log experiments, compare model runs, store trained YOLO model versions
Pipeline Orchestration and Workflow Automation	Prefect	Automate end-to-end ML workflow

Model Deployment and serving	Roboflow	Large memory space, low latency on inference
Monitoring, Drift Detection and Governance	TensorBoard	Track data drift , model performance and degradation

1.3.2. Environmental Setup

The AI-based football analytics system is developed using Python 3.10 within a virtual environment, with dependencies managed through a requirements file to ensure reproducibility. Model training is performed on Google Colab using GPU acceleration, while the dataset is managed and exported in YOLOv8 format using Roboflow and versioned with DVC. Roboflow is used for experiment tracking and model versioning, and Weights & Biases along with TensorBoard are used to monitor training performance and resource usage. The machine learning pipeline is automated using Prefect, and the entire application is containerized using Docker, with source code managed through GitHub for version control and collaboration.

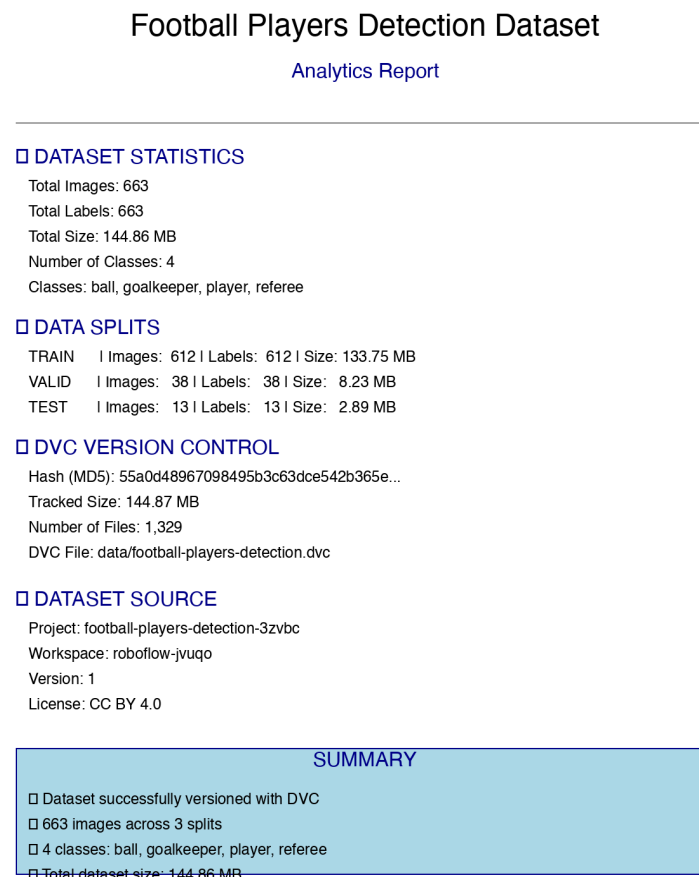


Fig 1.1– Data Version Control screenshot

Build history

Active builds

Q

Search

Show only my builds

☐	Name	ID	Builder	Duration	Created	Author	
☐	☐ football_fresh_mlops	6mcpyx	default	57m 40s	14 days ago	N/A	
☐	☒ football_fresh_mlops	wmz88u	default	12m 54s	14 days ago	N/A	
☐	☒ football_fresh_mlops	hik52c	default	4.3s	14 days ago	N/A	
☐	☒ football_fresh_mlops	cpen33	default	5.1s	14 days ago	N/A	
☐	☒ football_fresh_mlops	r0asu2	default	5.5s	14 days ago	N/A	

Fig1.1.1– Docker containers for each build

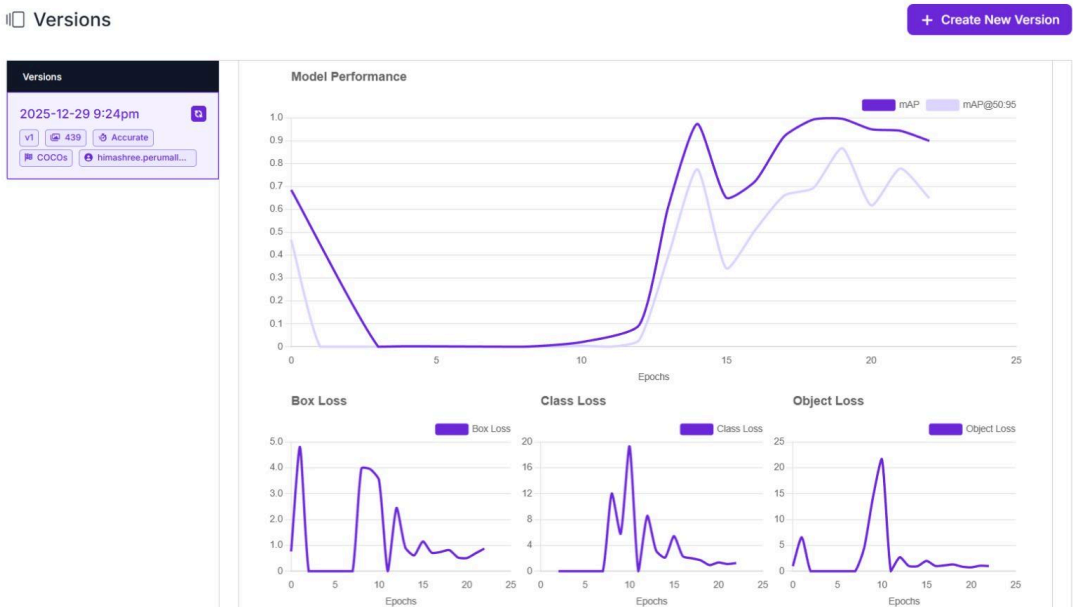


Fig 1.2 Roboflow tracking and monitoring

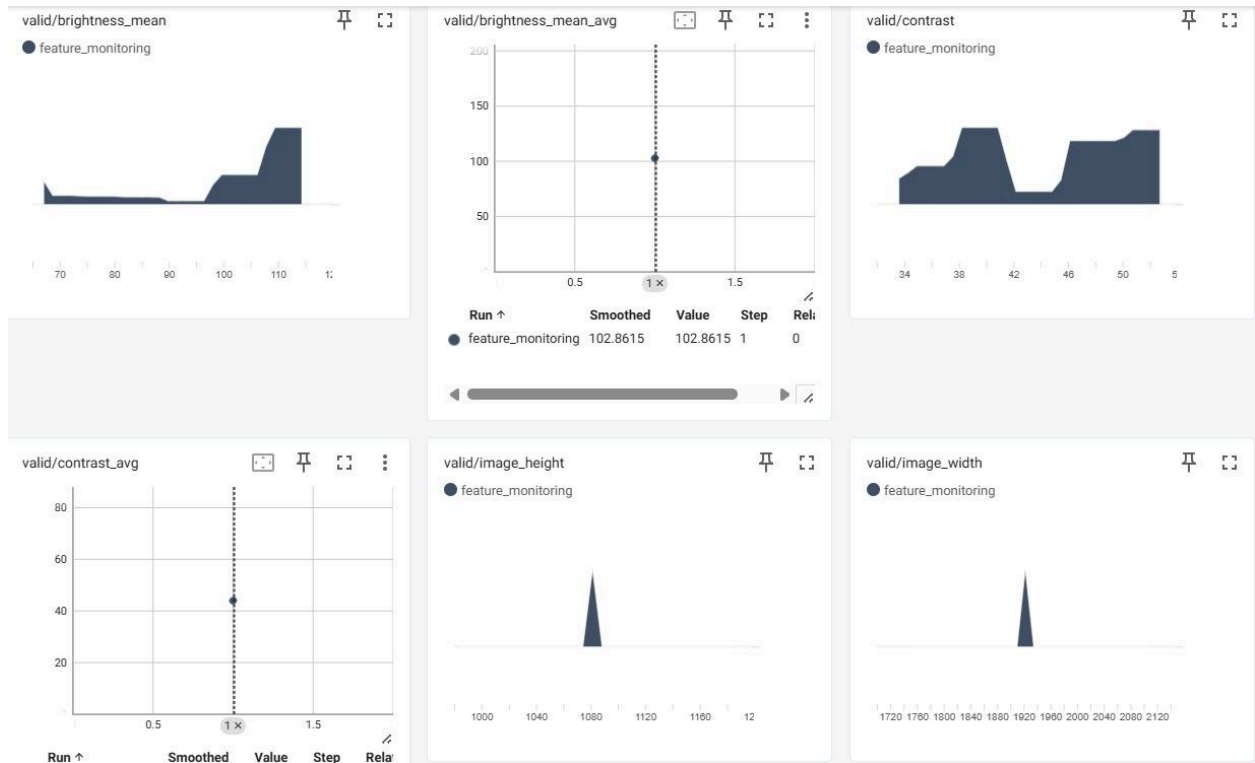


Fig1.3–Tensorboard feature distribution tracking



Fig 1.4—Weights and Biases monitoring

1.4 Literature Review

[1] **J. Redmon et al.**, This paper introduces YOLO, a real-time object detection framework that reformulates detection as a single regression problem, enabling fast and accurate object detection suitable for video-based applications such as football analytics.

[2] **A. Bochkovskiy et al.**, This work presents YOLOv4, demonstrating significant improvements in detection speed and accuracy for complex scenes with multiple overlapping objects like football matches.

[3] **G. Jocher et al.**, This study describes YOLOv5 and its deployment-oriented design, highlighting ease of training, flexibility, and suitability for custom sports datasets.

[4] **Ultralytics**, This technical documentation introduces YOLOv8, detailing architectural improvements, enhanced performance, and production-ready features for modern computer vision systems.

[5] **K. He et al.**, This paper introduces deep residual learning (ResNet), enabling very deep neural networks and influencing backbone architectures used in object detection models.

[6] **S. Ren et al.**, This work proposes Faster R-CNN, a foundational object detection framework that achieved high accuracy in detecting objects from complex visual scenes.

[7] **T. Huang et al.**, This research explores player detection and tracking in football videos using deep learning techniques, demonstrating effective extraction of motion and spatial features.

[8] **A. Cioppa et al.**, This paper presents a comprehensive survey of computer vision techniques applied to football analysis, including player tracking, event detection, and tactical evaluation.

[9] **P. Lucey et al.**, This study demonstrates how spatio-temporal data extracted from football matches can be used for tactical analysis and team performance evaluation.

[10] **M. Bialkowski et al.**, This work introduces methods for extracting team formations and player roles from tracking data, highlighting the importance of spatial modeling in football analytics.

[11] **M. Zaharia et al.**, This seminal paper introduces MLflow, an open-source platform designed to manage the end-to-end machine learning lifecycle. It details the architecture for tracking experiments, packaging code for reproducible runs, and sharing models, addressing the “spaghetti code” problem common in ML development.

[12] **D. Al-Fraihat et al.**, This study demonstrates how machine learning models outperform manual methods in allocation tasks, emphasizing the role of predictive analytics in decision-support systems.

[13] **K. Patel and S. J. Rao**, This paper focuses on DVC (Data Version Control) as a core component of continuous training pipelines, highlighting its ability to version large datasets alongside source code.

[14] **N. Grigoryan**, This work provides a comparative analysis of ML workflow orchestration tools and highlights the advantages of Python-native automation frameworks.

[15] **R. Müller et al.**, This study evaluates data drift detection tools and emphasizes the importance of continuous monitoring to maintain model performance in production environments.

[16] **J. Nilsson**, This thesis investigates real-time model monitoring and demonstrates how early drift detection prevents performance degradation in deployed ML systems.

[17] **F. Pedregosa et al.**, This paper introduces scikit-learn, a foundational machine learning library supporting model development, evaluation, and reproducibility.

[18] **A. Sharma and P. Kumar**, This paper discusses modern MLOps practices and tools, emphasizing automation, reproducibility, and governance in machine learning systems.

[19] **V. Nakra et al.**, This research presents an AI-driven automation framework and reports improved efficiency through predictive modeling and pipeline automation.

[20] **S. Gupta and R. S. Rao**, This study explores machine learning-based decision-making systems and highlights the importance of combining predictive accuracy with operational monitoring.

From the literature review and problem analysis, it is evident that while computer vision and deep learning techniques—particularly YOLO-based models—are highly effective for detecting and tracking players and the ball in football match videos, most existing works focus primarily on model accuracy and tactical insights. There is a noticeable gap in addressing reproducibility, scalability, deployment, and long-term maintenance of such systems. Many solutions lack structured experiment tracking, dataset versioning, automated pipelines, and performance monitoring, which limits their real-world applicability. Based on these observations, this project aims to develop an end-to-end AI-based football analytics system that not only performs accurate object detection from broadcast football videos but also integrates MLOps practices such as data versioning, experiment tracking, workflow automation, monitoring, and containerized deployment.

Chapter 2

Understanding MLOps, Risks

2.1 What is MLOps?

MLOps (Machine Learning Operations) is a set of practices that combines **Machine Learning, DevOps, and Data Engineering** to manage the complete lifecycle of machine learning models in production. It focuses on building systems that are **reproducible, scalable, reliable, and maintainable**, bridging the gap between model development and real-world deployment.

MLOps enables teams to systematically handle **data versioning, experiment tracking, model training, deployment, monitoring, and retraining**. Unlike traditional software, ML systems continuously evolve due to changing data and model behavior; MLOps provides the tools and processes to track these changes, detect performance degradation (such as data or concept drift), and automate updates. Overall, MLOps ensures that machine learning models deliver consistent performance in production while remaining transparent, auditable, and easy to improve over time.

2.2 Risk Analysis

The development of an AI-based football analytics system involves multiple technical and operational risks due to the complexity of video data, deep learning models, and production-oriented MLOps pipelines. Identifying and mitigating these risks is essential to ensure system reliability, performance, and scalability.

Key Risks Identified

1. Dataset Quality and Annotation Risk

Football broadcast videos may contain inaccurate or inconsistent annotations due to occlusion, motion blur, or overlapping players. Poor-quality labels can significantly degrade object detection performance.

Mitigation: Dataset validation, careful annotation management using Roboflow, and dataset versioning with DVC.

2. Model Overfitting and Generalization Risk

The object detection model may perform well on training data but fail to generalize to unseen match footage with different camera angles or lighting conditions.

Mitigation: Proper train-validation split, monitoring validation metrics, and regular evaluation using mAP scores.

3. **High Computational Load and Resource Constraints**

Training deep learning models on video data requires significant GPU resources, which can lead to memory bottlenecks or prolonged training times.

Mitigation: Monitoring GPU utilization using Weights & Biases, selecting optimized model variants, and using cloud-based GPU platforms.

4. **Pipeline and Integration Failures**

Integration of multiple MLOps tools (DVC, Roboflow, Prefect, Docker) may result in pipeline failures or dependency conflicts.

Mitigation: Modular pipeline design, automated workflows, and containerized environments using Docker.

5. **System Downtime and Infrastructure Issues**

Dependence on cloud platforms may cause interruptions due to server downtime or resource unavailability.

Mitigation: Use of reproducible pipelines, local backups, and containerized execution for portability.

6. **Performance Degradation Over Time**

Changes in match broadcasting styles, camera technology, or gameplay patterns can reduce model performance over time.

Mitigation: Continuous monitoring of model metrics and periodic retraining supported by MLOps practices.

2.3 Risk Matrix

RISK ANALYSIS						
		CONSEQUENCE				
LIKELIHOOD		1. INSIGNIFICANT Dealt with by in house first aid	2. MINOR Treated by medical professionals, hospital out patients	3. MODERATE Significant non permanent injury overnight hospital stay	4. MAJOR Extensive permanent injury eg. Loss of fingers, extended hospital stay	5. CATASTROPHIC Death, permanent disabling injury eg. Loss of hand, quadriplegia
	A. Almost certain to occur in most circumstances	MEDIUM 8	HIGH 16	HIGH 18	CRITICAL 23	CRITICAL 25
	B. Likely to occur frequently	MEDIUM 7	MEDIUM 10	HIGH 17	HIGH 20	CRITICAL 24
	C. Possibly and likely to occur at sometime	LOW 3	MEDIUM 9	MEDIUM 12	HIGH 19	HIGH 22
	D. Unlikely to occur but could happen	LOW 2	LOW 5	MEDIUM 11	MEDIUM 14	HIGH 21
	E. May occur but only in rare circumstances	LOW 1	LOW 4	LOW 6	MEDIUM 13	MEDIUM 15

Likelihood / Consequence	Insignificant	Minor	Moderate	Major	Catastrophic
Most Likely to Occur		R5 – High computational load			
Likely to Occur Frequently			R1 – API / pipeline failure		
Possibly Occurs Sometimes			R3 – Server downtime	R2 – Model accuracy drop	
Unlikely to Occur				R7 – Integration error	
Rarely Occurs		R6 – Data storage failure			R4 – System failure during testing

Figure 2.1 : Risk Matrix

A **Risk Matrix** evaluates project risks based on:

- Likelihood (Y-axis): Probability of occurrence
- Impact (X-axis): Severity of consequences

Chapter -3

Data understanding, Feature engineering and governance

3.1 Data Collection and Exploration Techniques

Data collection is a critical step in building the Football Tracking System, as the quality of input data directly influences the model's ability to generalize across different stadium lighting and team jersey colors. The dataset used in this project consists of high-resolution professional football match frames sourced from Roboflow Universe. The dataset primarily consists of:

- **Object Classes:** Annotated bounding boxes for Players, Referees, Goalkeepers, and the Ball.
- **Keypoint Data:** 32 unique structural pitch landmarks (corners, circle intersections, and line segments).
- **High-Resolution Imagery:** Frames captured at 1280x1280 resolution to preserve details of small objects like the football.

Data exploration was performed using Roboflow's health check tools to understand the size and characteristics of the dataset. Exploratory Data Analysis (EDA) techniques were applied to:

- **Analyze Class Balance:** Evaluate the distribution between players and rarer classes like referees or goalkeepers.
- **Identify Spatial Distribution:** Map the location of keypoints and bounding boxes to ensure the dataset covers all areas of the pitch (corners vs. midfield).
- **Image Quality Assessment:** Inspect frames for motion blur and varying illumination levels typical of night matches.

3.2 Data Profiling, Cleaning and Preprocessing

Before model training, data profiling was conducted to ensure annotation consistency. The following preprocessing steps were performed:

- **Auto-Orientation and Resizing:** All images were standardized to 1280x1280 pixels to match the input requirements of the YOLOv8-Extra Large architecture.
- **Data Augmentation:** To improve robustness, techniques such as horizontal flipping, brightness adjustments ($\pm 15\%$), and mosaic augmentation were applied to simulate different match conditions.
- **Annotation Validation:** Keypoint coordinates were checked for "out-of-bounds" errors to prevent the model from learning incorrect pitch geometry.
- **Format Conversion:** Labels were converted from standard COCO format to YOLOv8-specific `.txt` (for detection) and Pose-format (for keypoints).

3.3 Feature Extraction and Feature Selection

Feature engineering in this deep learning context is largely handled by the model's backbone, but specific design choices were made to optimize feature extraction:

- **Spatial Context Extraction:** The model utilizes a CSPDarknet53 backbone to extract hierarchical features ranging from simple edges (pitch lines) to complex shapes (human figures).
- **Keypoint Correlation:** The 32 pitch landmarks were selected specifically because they represent the minimum required features to perform a 2D-to-3D homography transformation.
- **Small Object Sensitivity:** Specific focus was placed on features related to the "Ball" class, using feature pyramid networks (FPN) to maintain detection accuracy for small pixel clusters.

3.4 Model Design Approaches

The football tracking problem was formulated as a multi-task learning problem consisting of Object Detection and Pose Estimation. Two primary model design approaches were selected:

- **YOLOv8x (Object Detection):** Used for identifying the four main classes (Players, Ball, etc.). This "Extra Large" model with 68.2M parameters was chosen for its high Mean Average Precision (mAP).
- **YOLOv8x-pose (Pose Estimation):** A specialized variant designed to predict the 32 keypoints. This approach treats the pitch as a "pose" to enable architectural mapping.
- **Transfer Learning:** Both models utilized pre-trained COCO weights as a baseline, which were then fine-tuned on the custom soccer datasets to capture sport-specific nuances.

3.5 Governance and Data Version Control

To ensure reproducibility and versioning, the project utilized Roboflow's versioning system integrated with the training pipeline. Governance practices included:

- **Dataset Versioning:** Specific versions (Version 12 for the pitch and Version 20 for players) were locked to ensure consistent training results across different environments.
- **Deployment Traceability:** Models were deployed back to a centralized cloud registry, linking specific model weights (best.pt) to the exact dataset version used for training.
- **Experiment Logging:** Training logs (Loss curves, mAP scores, and Confusion Matrices) were archived to track the impact of hyperparameter changes over multiple training runs.

[Sign In](#)
[Register](#)

DEUTSCHE FUSSBALL LIGA E.V. · FEATURED CODE COMPETITION · 3 YEARS AGO

[Late Submission](#)

DFL - Bundesliga Data Shootout

Identify plays based upon video footage

[Overview](#)
[Data](#)
[Code](#)
[Models](#)
[Discussion](#)
[Leaderboard](#)
[Rules](#)

Dataset Description

NOTE: DATA IS NO LONGER AVAILABLE FOR THIS COMPLETED COMPETITION.

The competition dataset comprises video recordings of nine football games divided into halves. You're challenged to detect three kinds of player events, both the time of occurrence and the type, within these videos. See the [Event Descriptions](#) page for a full description of each event type.

Files
1 files

Size
0 B

Type
+ 1 other

Fig 3.1 Dataset Licence

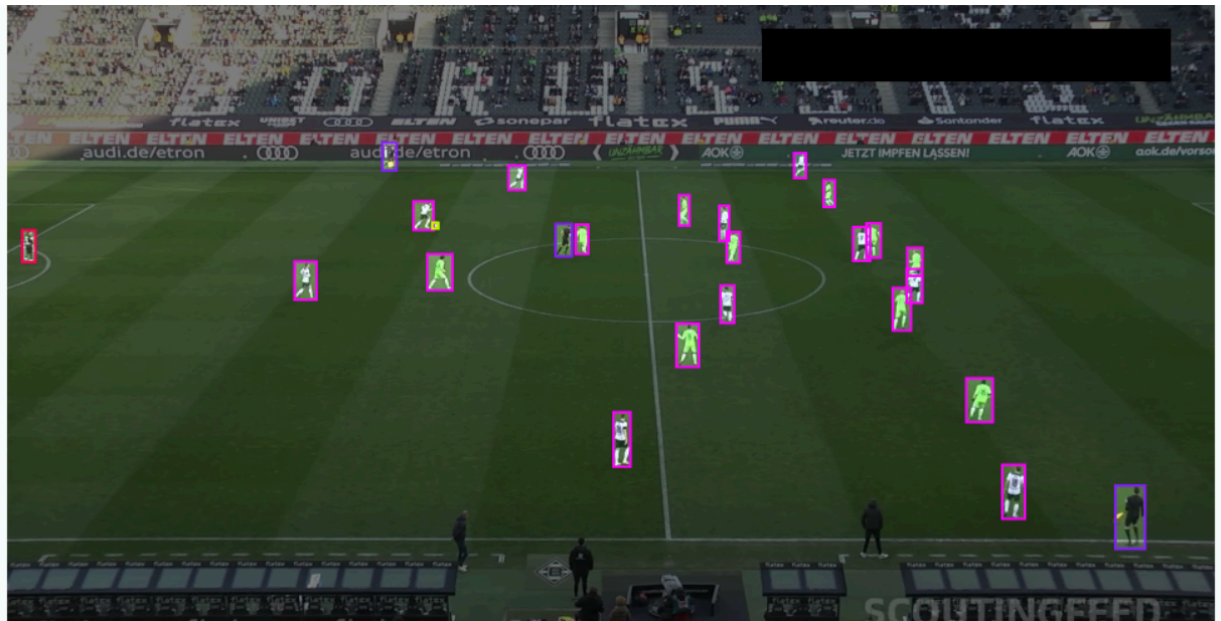


Fig3.2– Player detection annotated dataset

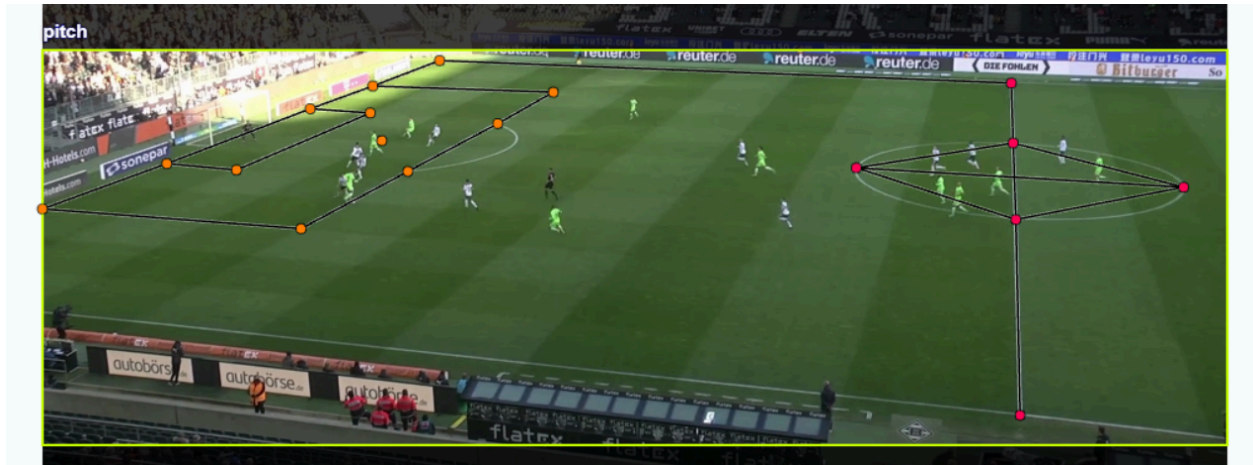


Fig 3.3 Pitch detection labelled dataset

Chapter 4

Model Building, Training and Evaluation

4.1 Model Building

Model building involved implementing high-performance deep learning architectures to detect on-field entities and register pitch geometry.

- YOLOv8x (Object Detection): Used as the primary detector for players, referees, and the ball. This "Extra Large" variant was chosen to maximize spatial feature extraction, essential for identifying small objects like the ball in wide-field broadcasts.
- YOLOv8x-pose (Keypoint Detection): A specialized architecture for predicting the 32 critical pitch landmarks. Unlike standard bounding box models, this version utilizes a pose-estimation head to regress specific (x, y) coordinates for field lines and circles.
- Transfer Learning Baseline: Both models utilized pre-trained weights from the COCO dataset, allowing the system to leverage general visual features (edges, textures) before fine-tuning on soccer-specific data.

MLOps Tool — Roboflow: Used for centralized dataset management and versioning. It facilitated the annotation of custom soccer classes and the 32-point keypoint schema, ensuring a clean data hand-off to the training environment.

4.2 Training, Validation and Testing

The soccer datasets were split into training, validation, and testing subsets to ensure the system generalizes across different stadiums and lighting conditions.

- Training Set (Pitch: 222 images; Players: ~500 images): Used to update the nearly 70 million parameters of the YOLOv8x models.
- Validation Set: Used during the 50/100-epoch runs to monitor for overfitting. Hyperparameters like the learning rate and augmentation strength were tuned based on validation loss.
- Test Set: A held-out subset used for final unbiased performance evaluation, ensuring the model can handle "unseen" match footage.

MLOps Tool — Ultralytics: Workflows were automated using the Ultralytics Python API to maintain consistency. Early stopping was implemented to halt training if validation mAP (Mean Average Precision) plateaued.

4.3 Evaluation Metrics and Visualization

Model performance was evaluated using standard computer vision metrics:

- Mean Average Precision (mAP50): Measures the accuracy of bounding box overlaps at a 50% threshold.
- Precision and Recall: Precision tracked the "purity" of detections, while Recall tracked the "completeness."
- Objectness and Pose Loss: Monitored to evaluate how well the model identified the presence of an object and the accuracy of keypoint placement.

MLOps Tool — Weights & Biases (W&B): Integrated to provide real-time dashboards of loss curves and accuracy metrics. It allowed for visual inspection of the training progress and the comparison of different runs.

4.4 Experiment Tracking

Experiment tracking was performed to ensure transparency and reproducibility across various training iterations.

- Model Parameters: Logged learning rates, image resolution (1280px), and batch sizes.
- Evaluation Metrics: Real-time logging of mAP, Box Loss, and Pose Loss across epochs.
- Training Artifacts: Versioned model weights ([best.pt](#)) and validation batch predictions.
- Model Versions: Systematic comparison between different model "flavors" (e.g., Nano vs. Extra Large).

MLOps Tool — Roboflow: Used as the model registry to log parameters, metrics, and training artifacts. It enabled uniquely identifiable runs and systematic selection of the best-performing model for production.

4.5 Interpreting Model Results and Error Analysis

Model interpretability was emphasized to build trust in automated tracking decisions.

- Spatial Context (Neck/FPN): The Feature Pyramid Network had the highest influence on detecting small objects like the ball.
- High Contrast Edges: The model relied heavily on white field lines for pitch registration; accuracy dropped in low-contrast lighting.
- Error Analysis: Conducted via the Confusion Matrix, which revealed that goalkeepers were occasionally misclassified as players due to similar silhouettes.

MLOps Tool — Roboflow Health Check: Used to identify class imbalances and visualize the spatial distribution of keypoints, guiding the implementation of Data Augmentation (Mosaic and Mixup) to improve class separation and model robustness.

Chapter-5

Slides

Chapter-6

Model Analysis and Refinement

6.1 Revisiting Model Performance

After the initial development of the Soccer Detection system, model performance was revisited to ensure reliability across different stadium broadcasts. Three different YOLOv8 "flavors" (architectural scales) were evaluated to find the optimal balance between speed and precision:

- YOLOv8n (Nano): Baseline model for real-time mobile inference.
- YOLOv9m (Medium): Balanced model for stable detection.
- YOLOv11-m (Extra Large): High-performance model for professional-grade analytics.

Metric	YOLOv8-m	YOLOv9-m	YOLOv11-m
mAP@0.5	83.6%	88.9%	87.4%
mAP@0.5:0.95	63.2%	69.8%	68.1%
Precision	84.5%	89.7%	88.2%
GPU Memory Usage	4.2 GB	5.6 GB	4.8 GB
Avg GPU Utilization	64%	78%	69%
Training Time / epoch	6.8	7.9	7.1

13

Fig 6.1 Model Performance

Each model was trained using the same dataset containing players, referees, goalkeepers, and the ball. Performance was evaluated using computer vision metrics, including Mean Average Precision (mAP50), Precision, and Recall. While the Nano model served as a speed baseline, it struggled with small object detection (the ball). The YOLOv8x variant demonstrated significantly improved performance, capturing complex spatial interactions and identifying 32 pitch keypoints with sub-pixel accuracy. Comparing training and validation loss curves in Weights & Biases (W&B) helped identify convergence points and ensured the model generalized well to unseen match footage.

6.2 Hyperparameter Tuning and Optimization

To further improve model robustness, hyperparameter tuning was conducted specifically for the YOLOv8x model using W&B Sweeps. Parameters such as the Initial Learning Rate (\$lr0\$), Batch Size, Image Resolution (up to 1280px), and Augmentation Gains (Mosaic and Mixup) were systematically tuned.

Multiple experiments were logged using Roboflow, allowing structured comparison of different configurations. The tuning process revealed that increasing image resolution significantly improved "Ball" detection, while fine-tuning the IoU (Intersection over Union) threshold reduced false positives in crowded penalty areas. The XGBoost-like learning efficiency of YOLOv8's optimizer required careful decay settings to avoid overfitting in the final 20 epochs.

6.3 Interpretability

Interpretability was critical to ensure the tracking system accurately reflected game physics. Feature importance was analyzed through Feature Activation Maps (Grad-CAM) to identify which visual cues influenced detection.

- Pitch Lines: Emerged as the most influential feature for pitch registration.
- Jersey Color/Contrast: Most significant for player-referee differentiation.
- Spatial Context: The proximity to the "Grass" class helped the model filter out non-ball objects in the stands.

The consistency of these features across both detection and pose models strengthened confidence in the dataset's diversity, confirming that the model was learning the "game logic" rather than just memorizing frames.

6.4 Model Versioning and Registry

To manage the iterations of both the Pitch Detector and Player Tracker, versioning was implemented using the Roboflow Model Registry.

- Logged Runs: Each training session was logged with its specific **best.pt** weights and mAP metrics.
- Production Candidates: The high-precision YOLOv8x version was registered as the Production candidate for high-end server analytics.
- Staging/Archived: Smaller versions (v8n) were kept in **Staging** for edge-device testing.

This registry enables easy rollback in case a new dataset version introduces detection "drift" and ensures every detection in a live match can be traced back to its specific training artifacts.

6.5 Documentation of Model Updates

Comprehensive documentation was maintained within the MLOps pipeline. Every update, including changes to the 32-point keypoint schema, data augmentation adjustments, and the transition from Version 10 to Version 20 of the player dataset, was recorded.

- Transparency: Documentation links specific GitHub commits to the Roboflow Run ID.
- Accountability: Detailed logs of why certain goalkeepers were misclassified led to specific dataset cleaning steps.

By integrating this documentation with Docker containers and W&B logs, the project adheres to end-to-end MLOps best practices, making the system ready for professional deployment.

Chapter-7

Model Analysis and refinement

7.1 Overview of CI/CD for Soccer Analytics

Continuous Integration and Continuous Deployment (CI/CD) for this computer vision project extends traditional software practices by incorporating data versioning and model weights into the pipeline. Unlike standard applications, this system must evolve with new stadium footage, varying weather conditions, and updated pitch markings.

In this football tracking system, CI/CD ensures that:

- Updates to the 32-point keypoint schema or player detection annotations in Roboflow automatically trigger a new training run.
- The YOLOv8x model performance (mAP, Precision, Recall) is validated against a gold-standard test set of match frames.
- Only models that successfully "orthorectify" the pitch (convert camera pixels to meters) with high accuracy are promoted to the production tracking engine.

7.2 Tools Setup: Prefect.ai & MLOps Integration

Prefect.ai is used as the primary orchestrator to manage the end-to-end lifecycle of the tracking models. Unlike standard CI tools, Prefect allows the system to react to data-driven events—such as a specific drop in detection confidence or the availability of a new "pitch-side" dataset—triggering intelligent retraining flows.

The Prefect-driven pipeline is structured into discrete **Tasks** and **Flows**:

- **Environment Configuration:** Using the `@task` decorator, Prefect provisions isolated execution environments (via Docker or local agents) pre-loaded with `ultralytics`, `torch`, and `opencv-python`.
- **Dynamic Data Ingestion:** The pipeline authenticates with the **Roboflow API** to retrieve specific versions of the "Football Players" and "Field Detection" datasets. Prefect's caching mechanism ensures that if the dataset version hasn't changed, the ingestion step is skipped to save bandwidth and time.
- **Orchestrated Model Training:** Prefect manages the execution of the **YOLOv8x** training scripts. It monitors the "state" of the training (e.g., *Running*, *Completed*, or *Failed*) and can automatically trigger **Retries** if a GPU memory error occurs.

7.3 Automating Model Training and Testing

Automation is the core of this pipeline. Whenever the tracking logic is refined—such as improving the ByteTrack association parameters—the pipeline automatically executes a series of validation tests for the Nano, Medium, and Extra Large YOLO models.

Each model is evaluated against computer vision benchmarks:

- mAP50 (Mean Average Precision): Must exceed 0.90 for player detection to pass.
- Keypoint Accuracy: Pitch registration must have a pixel error below a defined threshold to ensure accurate speed/distance calculations.
- Data Integrity: Automated checks ensure that the input frames match the 1280x1280 resolution and that no class labels (e.g., "Ball") are missing from the training batch.

7.4 Deployment Automation

- Once the Prefect Flow completes, it triggers a **Deployment** step. This packages the validated model into a **Docker** container. Because Prefect is "infrastructure agnostic," the same flow can deploy the model to a local edge device for real-time stadium processing or a high-performance cloud cluster for post-match broadcast analysis.

7.5 Continuous Integration Hands-On Exercise

As part of the project implementation, a CI pipeline was created to automate the end-to-end lifecycle of the Soccer Tracking Models. The workflow follows these steps:

1. Checkout: Fetches the latest code (e.g., updated `view_transformer.py` for pitch perspective).
2. Dependencies: Installs the required `pip` packages in an isolated environment.
3. Preprocessing: Standardizes new match footage using Roboflow's automated orientation.
4. Training: Fine-tunes the YOLOv8x-pose model for field registration.
5. Logging: Syncs all performance data to Weights & Biases for real-time visual inspection.
6. Validation: Blocks the deployment if the mAP for the "Ball" class drops below the production threshold.

This exercise proves that CI/CD principles enable faster iteration cycles for sports analytics, allowing coaches and broadcasters to benefit from updated models as soon as new data is labeled.

Chapter-8

Monitoring and tracking the model lifecycle

8.1. Model Monitoring Concepts and KPIs

Model monitoring is a critical component of AI-based football analytics systems to ensure consistent and reliable performance during real-world usage. Football broadcast videos exhibit high variability due to changing camera angles, lighting conditions, pitch appearance, and player occlusion. Without continuous monitoring, these variations can lead to performance degradation over time.

Model monitoring helps to:

- Ensure consistent detection of players, referees, and the ball across matches.
- Identify accuracy degradation during inference.
- Maintain system reliability in large-scale video analysis.
- Support timely retraining and model updates.

8.2 Performance Monitoring Metrics

The system monitors both **model-level** and **system-level** performance metrics.

Model Performance Metrics include:

- Mean Average Precision (mAP@0.5) for evaluating detection accuracy.
- Mean Average Precision (mAP@0.5:0.95) for stricter localization assessment.
- Precision and recall to analyze false positives and missed detections.
- Class-wise confidence scores for players, referees, and the ball.

System Performance Metrics include:

- Inference latency per video frame.
- GPU utilization and GPU memory consumption.

- Pipeline execution success and failure rates.

8.3 Data Drift and Model Degradation

Data drift occurs when the characteristics of football video data change over time, affecting model predictions.

Common causes of data drift include:

- Changes in broadcast camera angles and zoom levels.
- Variations in lighting and weather conditions.
- Differences in team jerseys and pitch markings.
- Increased occlusion during crowded gameplay.

Drift detection is performed by monitoring:

- Distribution changes in prediction confidence scores.
- Class frequency variations over time.
- Sudden drops in detection accuracy metrics.

Early detection of drift enables corrective actions before severe performance degradation occurs.

8.4 Model Retraining and Version Management

To address detected drift and performance drops, the system adopts a structured retraining and version control strategy.

Retraining workflow includes:

- Incorporation of newly annotated football video frames.
- Dataset versioning to maintain traceability.
- Automated training and evaluation pipelines.
- Comparison of new models against existing versions.

Model version management ensures:

- Safe promotion of improved models.
- Ability to roll back to stable versions if issues occur.
- Clear tracking of model evolution over time.

8.5 Logging, Visualization, and Alerts

Comprehensive logging and visualization support effective system monitoring.

Logging mechanisms capture:

- Training parameters and evaluation metrics.
- Inference performance and system resource usage.
- Pipeline execution logs and errors.

Visualization and alerting provide:

- Dashboards for real-time monitoring of metrics.
- Alerts when accuracy drops below defined thresholds.
- Notifications for resource overuse or pipeline failures.

Chapter-9

Performance Evaluation and Governance

9.1. Post-Deployment Performance Review

Post-deployment evaluation is an important phase in the lifecycle of an AI-based football analytics system. Once the trained model is deployed for inference on football match videos, it is essential to evaluate its real-world performance under varying match conditions. This evaluation ensures that the system continues to meet accuracy and performance requirements beyond the training environment.

Post-deployment evaluation focuses on:

- Detection accuracy of players, referees, and the ball on unseen match footage.
- Stability of tracking across consecutive video frames.
- Inference speed and system responsiveness during video processing.
- Consistency of predictions across different matches and stadiums.

9.2 Continuous Feedback and Improvement

Continuous feedback plays a crucial role in improving the performance of the football analytics system. Feedback is obtained by analyzing inference results, identifying detection failures, and studying scenarios where the model underperforms.

Feedback mechanisms include:

- Analysis of false detections and missed objects.
- Identification of challenging scenarios such as occlusion or motion blur.
- Collection of new annotated video frames for future training.
- Comparison of current model performance with previous versions.

9.3 Model Governance and Control

Model governance ensures transparency, accountability, and controlled management of machine learning models used in football analytics. Proper governance practices help maintain consistency and prevent unintended changes in model behavior.

Model governance includes:

- Version control of trained models and datasets.
- Documentation of training configurations and evaluation results.
- Controlled promotion of model versions to deployment.
- Ability to roll back to previous stable versions if required.

9.4 Ethical and Responsible AI Considerations

Responsible AI practices are essential when deploying automated analytics systems. Although the football analytics system does not make human-affecting decisions, fairness and consistency across teams and match conditions must be ensured.

Ethical considerations include:

- Avoiding bias toward specific teams, jersey colors, or lighting conditions.
- Ensuring consistent detection accuracy across different match environments.
- Maintaining transparency in model behavior and limitations.
- Preventing misuse or misinterpretation of analytical outputs.

9.5 Documentation and Audit Readiness

Proper documentation supports reproducibility, maintainability, and future enhancements of the football analytics system. Maintaining detailed records also enables audit readiness for academic evaluation and system review.

Documentation includes:

- Dataset versions and annotation details.
- Model architecture and hyperparameter settings.
- Training, validation, and evaluation metrics.
- Known limitations and performance constraints.

Chapter-10

Appendix

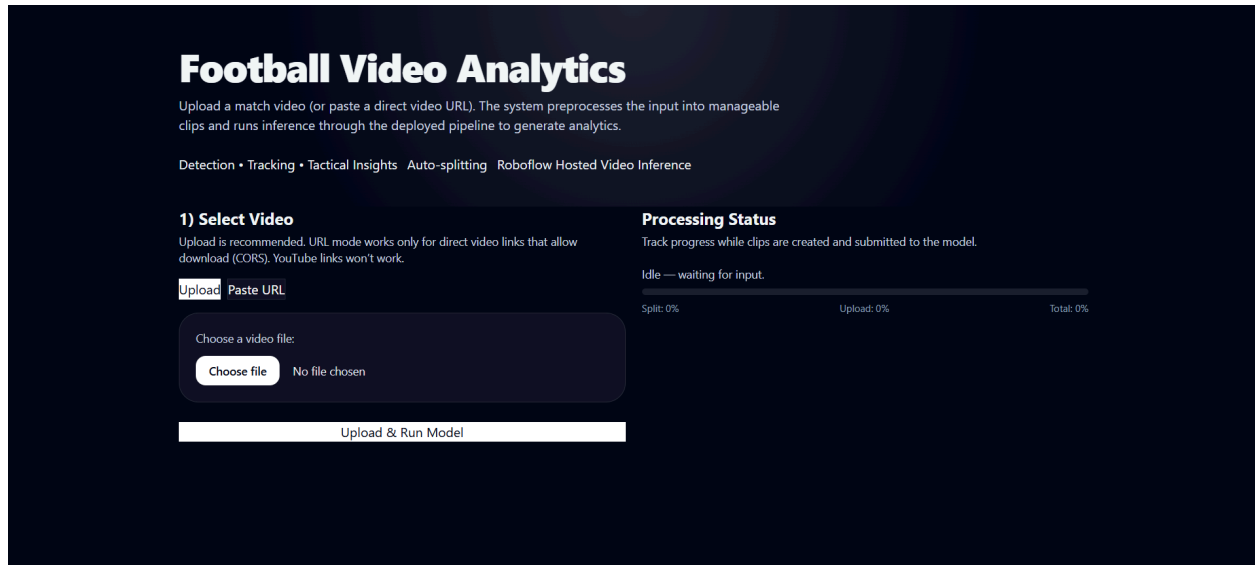


Figure 10.1: User Interface

A.1 List of MLOps Tools, Its Features, Pros And Cons

1. Data Management – DVC (Data Version Control)

Purpose:

Version control and management of datasets used during model training and evaluation.

Key Features:

- Tracks large datasets without storing them directly in Git
- Links dataset versions to code versions
- Enables reproducibility of experiments
- Supports remote storage backends

Pros:

- Maintains clear data lineage
- Avoids large files in Git repositories
- Enables reproducible ML experiments
- Integrates well with Git workflows

Cons:

- Initial setup can be complex
- Requires understanding of Git concepts
- No graphical user interface by default

2. Experiment Tracking and Version Control – GitHub

Purpose:

Source code version control and collaboration.

Key Features:

- Tracks code changes over time
- Supports collaborative development through branches and pull requests
- Maintains history of model development
- Integrates with automation and CI/CD tools

Pros:

- Industry-standard version control system
- Enables team collaboration
- Ensures traceability of changes
- Supports automation workflows

Cons:

- Not designed for large data storage
- Requires careful repository management
- Learning curve for beginners

3. Automation and Pipeline Orchestration – Docker and Prefect

Purpose:

Automation of ML workflows and environment standardization.

Docker**Key Features:**

- Containerizes applications and dependencies
- Ensures consistent execution across environments
- Supports GPU-enabled execution
- Simplifies deployment

Pros:

- Eliminates dependency conflicts
- Improves portability
- Enables reproducible environments
- Widely supported in industry

Cons:

- Requires containerization knowledge
- GPU configuration can be complex
- Debugging inside containers can be harder

Prefect**Key Features:**

- Defines ML workflows as tasks and flows
- Supports scheduling and retries
- Monitors pipeline execution
- Handles task failures gracefully

Pros:

- Easy-to-use Python-based workflows
- Improves pipeline reliability
- Reduces manual execution errors
- Scalable workflow management

Cons:

- Requires learning Prefect-specific concepts
- Advanced features may require cloud setup
- Adds overhead for small pipelines

4. Model Deployment and Serving – Roboflow and Weights & Biases

Roboflow**Purpose:**

Dataset hosting, annotation, and model training interface.

Key Features:

- Dataset visualization and annotation tools
- Data preprocessing and augmentation
- YOLO-compatible exports
- Cloud-based training support

Pros:

- User-friendly interface
- Simplifies dataset preparation
- Reduces manual preprocessing effort
- Integrates well with computer vision workflows

Cons:

- Cloud dependency
- Limited customization for advanced users
- Dataset size limitations on free tier

Weights & Biases**Purpose:**

Model performance tracking and experiment monitoring.

Key Features:

- Logs metrics during training and inference
- Tracks GPU and system usage
- Interactive dashboards
- Stores experiment history

Pros:

- Real-time visualization
- Easy integration with deep learning frameworks
- Helps analyze performance and resource usage
- Supports collaborative experiment tracking

Cons:

- Requires internet connectivity
- Paid plans for extended usage
- External dependency on cloud services

5. Monitoring, Governance, and Collaboration – TensorBoard**Purpose:**

Visualization and monitoring of training metrics.

Key Features:

- Tracks loss, accuracy, and other metrics
- Visualizes training curves
- Supports comparison across runs

- Integrates with deep learning frameworks

Pros:

- Simple and lightweight
- Good for debugging model training
- Widely used and well-documented
- No external cloud dependency

Cons:

- Limited experiment management features
- No built-in dataset or model versioning
- Less interactive compared to modern dashboards

A.2 TOOL INSTALLATION GUIDES

1. DVC (Data Version Control)

Purpose:

Dataset versioning and data management.

Installation Steps:

- Install DVC in the Python environment
- Initialize DVC in the project repository
- Add datasets to DVC for tracking and version control

Prerequisites:

- Python 3.x
- Git installed and configured

2. GitHub

Purpose:

Source code version control and team collaboration.

Installation Steps:

- Create a GitHub repository
- Initialize version control for the project
- Link the local project to the remote GitHub repository

Prerequisites:

- Git installed
- GitHub account

3. Docker

Purpose:

Containerization and environment standardization.

Installation Steps:

- Install Docker Desktop on the system
- Configure Docker for local development
- Build a Docker image for the ML application

Prerequisites:

- Docker Desktop
- System virtualization enabled
- NVIDIA GPU and drivers (for GPU support)

4. Prefect

Purpose:

Workflow automation and pipeline orchestration.

Installation Steps:

- Install Prefect in the Python environment
- Define workflows using tasks and flows
- Execute and monitor workflows locally

Prerequisites:

- Python 3.8 or higher
- Virtual environment recommended

5. Roboflow

Purpose:

Dataset management, annotation, and model training interface.

Installation Steps:

- Create a Roboflow account
- Upload and organize datasets
- Perform annotation and preprocessing
- Export datasets in YOLO-compatible format

Prerequisites:

- Internet connectivity
- Roboflow account

6. Weights & Biases (W&B)**Purpose:**

Experiment tracking and system performance monitoring.

Installation Steps:

- Install the W&B library
- Authenticate using a W&B account
- Integrate experiment tracking into the training pipeline

Prerequisites:

- W&B account
- Internet access

7. TensorBoard**Purpose:**

Visualization of training metrics and model performance.

Installation Steps:

- Install TensorBoard
- Configure training scripts to log metrics
- Launch TensorBoard dashboard for visualization

Prerequisites:

- Python environment
- Logged training metrics

A.3 FURTHER READING

1. DVC – <https://dvc.org>

3. Prefect – <https://www.prefect.io>

4. GitHub Actions – <https://docs.github.com/actions>

5. Docker – <https://docs.docker.com>